视觉第五课：编译原理、代码规范、C++基础

> **免责声明：** 学长也只是比你们早来了一年电🧱，所以也会犯错误，如果发现了我的错误，请狠狠的指正我，谢谢 💐💐💐

编译原理

对于平常的应用程序及开发，我们常使用的 IDE，如微软公司的 Visual Studio、JetBrains 公司的 Clion 等，这样的编译和链接的过程一步完成，通常这种合并在一起的过程称为**构建**。但简单的一句 `gcc helloworld.c` ，就包含了非常复杂的过程。

现代编译器和 IDE 所集成的默认配置，编译和链接参数对于大部分的应用程序开发和工程已经相当够用。但是在这种开发过程中，我们往往由于这种 IDE 的娇生惯养，导致我们这些婴幼儿程序员在面临一些程序的性能瓶颈束手无策，因为不了解程序背后的本质，所以我们要学习编译原理，这对于一个 C++ 和其他 C 系语言以及他们的亲戚们（包括沙比缝合怪）的工程师尤为重要。

什么是编译器

被隐藏的过程

经典的 hello world 程序作为学习计算机的你来说闭着眼睛都能敲出来。

代码块

```c
#include <stdio.h>

int main(void){
    printf("hello world\n");
    return 0;
}

```

在 Windows 下我们在配置 gcc 后就可以通过：

代码块

```
gcc helloworld.c
```

就会产生了 `.exe` 文件产生结果；而在 Linux 系统下：

代码块

```
gcc helloworld.c
./a.out
```

结果：`hello world`

事实上经过了四个步骤：

1. **预处理（preprocessing）**：根据已放置的文件中的预处理指令来修改源文件的内容
2. **编译（compilation）**：通过词法分析和语法分析，在确认所有指令都是符合语法规则之后，将其翻译成等价的中间代码表示或汇编代码
3. **汇编（assembly）**：把汇编语言代码翻译成目标机器指令的过程
4. **链接（linking）**：找到所有用到的函数所在的目标文件，并把它们链接在一起合成为可执行文件（executable file）

预处理

预处理器是在程序源文件被编译之前，根据预处理指令对程序源文件进行处理的程序。预处理器指令以 `#` 号开头标识，末尾不包含分号。预处理命令不是 C/C++ 语言本身的组成部分，不能直接对它们进行编译和链接。C/C++ 语言的一个重要功能就是可以使用预处理指令和具有预处理的功能。C/C++ 提供的预处理功能主要有**文件包含、宏替换、条件编译**等。

预处理的命令

代码块

```
gcc -E helloworld.c -o hello.i
# -o + 输出的文件名

# 另外一种形式
cpp helloworld.c > helloworld.i
```

头文件包含

头文件是一种文本文件，使用文本编辑器将代码编写好之后，以扩展名 `.h`（`.hpp`）保存就行了。头文件中一般放一些重复使用的代码，例如函数声明、变量声明、常数定义、宏的定义等等。当使用预处理指令 `#include` 引用头文件时，相当于将头文件中所有内容，复制到 include 处。那么编译器在哪里找到 include 的头文件呢？这就涉及到了 include 的搜索机制。

头文件有两种：

代码块

```
#include <Library.h>
```

```
2   #include "Library.h"
```

这两种有什么区别呢？

- 在使用 `<>` 时，编译器会在标准系统目录中搜索指定文件。这些目录由编译器预定义，通常包含系统或库头文件。该语法通常用于包含标准库头文件。
- 使用 `""` 时，编译器会首先在当前目录下查找指定文件。如果在当前目录下找不到该文件，编译器就会在标准系统目录下搜索。这种语法通常用于包含作为项目或程序一部分的头文件。

系统目录一般是：

代码块

```
1   /usr/include
2   /usr/local/include
3   /usr/lib/gcc-lib/linux/11/include
```

每个人不一样的，别直接以为我的就是绝对的。

注意

对于用户自己编写的头文件，宜用双引号形式。对于系统提供的头文件，既可以用尖括号形式，也可以用双引号形式，它们都能找到被包含的文件，但显然用尖括号形式更直截了当，效率更高。但是有一些 C 语言书籍的笔者写的时候常常用 `""` ，这就是不好好学编译原理的原因，请大家不要养成不好的习惯。

宏替换

首先什么是宏？宏的格式如下：

代码块

```
1   #define <宏名/标识符> <字符串>
```

举例来说：

代码块

```
1   #define PI 3.14
```

说明

① 宏名一般用大写。

② 宏定义末尾不加分号。

③ 可以用 `#undef` 命令终止宏定义的作用域。

```c
#include <stdio.h>

#define SIZE 5

int main(void) {
    int a[SIZE];                // SIZE 被替换成 5
    printf("%d\n", SIZE);       // 输出 5

#undef SIZE                     // 从这行起，SIZE 不再是宏

    // int b[SIZE];             // 若直接这样用：编译报错 'SIZE' undeclared

#define SIZE 10                 // 可以重新定义（值不同也行）
    int c[SIZE];                // 此时 SIZE 替换成 10
    printf("%d\n", SIZE);       // 输出 10
    return 0;
}
```

④ 宏定义可以嵌套。

```c
#include <stdio.h>

#define PI 3.14
#define SQUARE(x)  ((x) * (x))
#define CIRCLE_AREA(r)  (PI * SQUARE(r))   /* 里面用了 PI 和 SQUARE — 嵌套 */

int main(void) {
    printf("%f\n", CIRCLE_AREA(2));
    return 0;
}
```

⑤ 字符串 `""` 中永远不包含宏。

⑥ 宏替换在编译前进行，不分配内存；变量定义分配内存；函数调用在编译后程序运行时进行，并且分配内存。

⑦ 预处理是在编译之前的处理，而编译工作的任务之一就是语法检查，预处理不做语法检查。

⑧ 使用宏可提高程序的通用性和易读性，减少不一致性，减少输入错误和便于修改。例如：数组大小常用宏定义。

编译

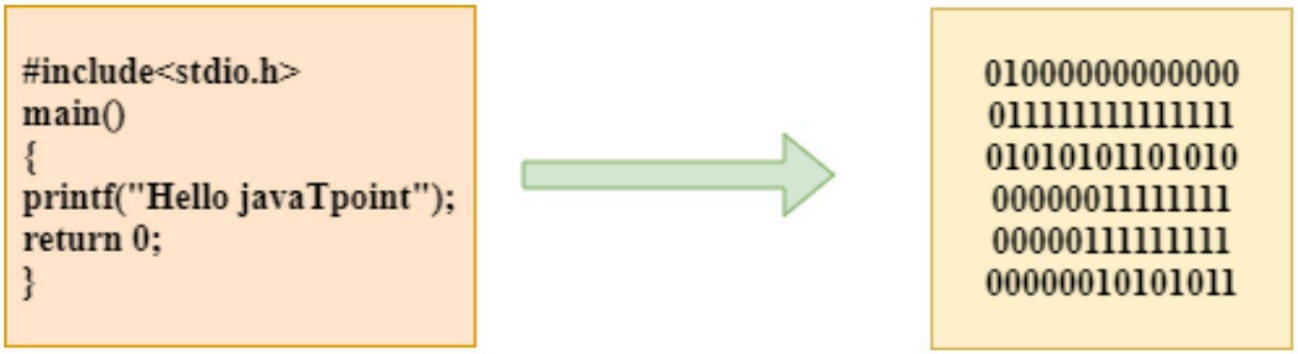
编译使用的命令是：

```
代码块
1   g++ -g -pedantic -Wall -o executable_file_name source_file_name.C
```

- `g++`：编译器
- `-g`：生成调试信息
- `-pedantic`：遵循 c/c++ 标准
- `-Wall`：(Warning all) 打开大部分常见警告
- `-o executable_file_name source`：输出文件名
- `source_file_name.C`：输入源文件

汇编

这里放一张图片可能更明显：



而在编译语言中，就是将预处理的文件经过汇编器转换为机器能够看懂的二进制文件；

我们可以使用 gcc 来帮助我们实现

```
代码块
1   gcc -c helloworld.s -o helloworld.o
```

也可以直接调用汇编器：

```
代码块
1   $as helloworld.s -o helloworld.o
```

链接

链接其实是这四步中最麻烦的，也是最不容易让人理解的部分链接其实就是从多个对象文件

(object file)来生成一个可执行文件(executable file)

为什么要有链接器呢?

在我们编写工程的时候，在编译过程中，如果编译器找不到某个函数的定义，它会简单地认为该函数是在另一个文件中定义的。

如果不是这种情况，编译器就会呆掉了它不会一次查看多个文件的内容。

但是，链接器可能会查看许多文件，并试图找到未提及的函数的引用。相反，它使用所谓的 "条件编译"。

因此，必须只编译那些有变化的源文件。至于其他文件，目标文件是链接器的适当输入。

最后，它还使预编译代码库的实施变得更加容易。只需创建对象文件，然后像链接其他对象文件一样链接它们即可。

为了充分发挥条件编译的优势，找一个程序来帮助你可能比试图记住上次编译后你修改了哪些文件要容易得多。

如果使用集成开发环境 (IDE)，就可以自己解决这个问题

结论

因此，了解编译阶段和链接阶段之间的区别至关重要。换句话说，这可以让查找错误变得更容易。

编译器错误一般都是语法错误—少了一个分号，多了一个括号。

另外，链接错误一般涉及缺失或多重定义。

如果从链接器中得到一个函数或变量被多次定义的错误信息，那么很可能是两个源代码文件中存

在相同的函数或变量。

代码规范

首先指出，我们今天讲的代码规范只是纠正大家刚开始写代码时的一些不良习惯，不涉及到代码

的风格问题，只要自己觉得舒服就行。

命名

变量命名

代码块

```
1   int a,b,c,d,e,f;// 单字母变量
```

```
int a1,a2,a3,a4;// 含糊不清的变量名
int shuzu[10],wei;// 拼音变量名
int minwei;// 拼音+英文变量名
```

这些都是不合适的命名。变量名应该尽可能的清晰明了，让人一眼就能看出这个变量的作用。比如：

代码块

```
int temp; // 用于暂存的变量
int sum; // 用于求和的变量
float tempurature; // 温度
```

那么这里给大家提出一点要求：在命名变量时，要求变量名的长度不少于 3 个字符，且要求变量名能够清晰的表达出变量的作用。例如这样的命名也是不合适的：

代码块

```
int aaa,bbb,ccc,ddd,eee,fff;
int qwe;
int jflskgjwe;
```

当然对于约定俗成的变量名，如循环变量 i，j，k 等，可以就用其约定俗成的变量名。

函数命名

函数命名的要求和变量命名的要求类似，要求尽可能的清晰明了，让人一眼就能看出这个函数的作用。比如：

代码块

```
vcoid printArray(int *array, int length); // 打印数组
void swap(int *a, int *b); // 交换两个数
void quickSort(int *array, int left, int right); // 快速排序
```

同样的，函数名的长度也要求不少于 3 个字符，且要求函数名能够清晰的表达出函数的作用。

注释

C 语言中的注释有两种：

代码块

```
// 单行注释，从 // 开始到行末都是注释，又称行注释

/*
* 多行注释
* 从 / * 开始到 * / 都是注释
* 又称块注释
*/
```

注释的作用是让人更容易理解代码，所以注释的内容应该是对代码的解释，而不是对代码的重复。

比如：

代码块

```
void swap(int *a, int *b) { // 交换两个数

    int temp = *a;// 存下 a 的值

    *a = *b;// 将 b 的值赋给 a

    *b = temp;// 将 a 的值赋给 b

}

```

缩进、空格

在 C 语言中，缩进和空格都是没有语法意义的，但是缩进和空格能够让代码更加清晰明了，所以

我们要求大家在写代码时必须缩进，必须使用空格。

为什么不使用 Tab 键?

因为不同的编辑器对Tab 键的处理方式不同，有的会将 Tab 键转换为空格，有的会将 Tab 键转换为 4 个空格，有的会

将 Tab 键转换为 8 个空格。同时，Tab 有时候会导致代码在不同的编辑器中显示不一致，所以我们建议大家使用空格。**ps: 不是手打空格，而是建议将 Tab 设置成 4 个空格。例如下面的代码就是不合格的:**

```
// 没有缩进

int adder(int a, int b) {

int sum;

sum = a + b;

return sum;

}

// 缩进不统一

void adder(int *a, int *b) {

    int sum = 0;

        for(int *p = a; p <= b; p++) {

        sum += *p;

    }

return sum;

}
```

此外，在运算符两边都要加上空格，逗号后面也要加上空格。如前面的代码。

未定义行为

自增自减运算符

在 C 语言中，有一些行为是未定义的，也就是说，这些行为的结果是不确定的，可能在不同的编译器中结果不同，也可能在同一个编译器中结果不同。例如，谭浩强老师的《C 程序设计》中就提到了这样一个例子：

代码块

```
1   int a = 1;
2   printf("%d,%d\n", a++, a++);
```

这段代码的输出结果是不确定的，可能是 1,2，也可能是 2,1，也可能是 1,1，也可能是 2,2。或者我们来看这个例子：

代码块

```
1   int a = 1;
2
3   int b = a++ + ++a;
4
5   printf("%d\n", b);
```

这段代码的结果也是不确定的，可能是 3，也可能是 4，也可能是 5。首先一点就是，**C 语言中自增运算符 ++ 和自减运算符 -- 每条语句中只能出现一次**。

函数声明

注意这段代码：

代码块

```
1   #include <stdio.h>int main(void)
2   {
3
4       int a = adder(1, 2);
5
6       printf("%d", a);
7
8       return 0;
9
10  }
11
12  int adder(a, b) {
13
14      return a + b;
15
```

```
16  }
```

在 gcc 下编译运行这段代码，会得到一个警告，但是程序仍然可以正常运行。原因是在未声明而使用函数 adder 时，编译器会默认函数的返回值为 int；未指定函数参数类型时，编译器会默认函数参数类型为 int。 如下面两段代码：

代码块

```
1   #include <stdio.h>
2   int main(void)
3   {
4       long a = adder(1, 2);
5
6       printf("%ld", a);return 0;
7   }
8
9   long adder(a, b) {
10
11      return a + b;
12
13  }
14  // 会编译报错，函数定义与函数声明不一致
15
16
17  #include <stdio.h>
18
19  float adder(a, b) {
20
21      return a + b;
22
23  }
24  int main(void){
25
26      float a = adder(1.1, 1.2);
27
28      printf("%f", a);
29
30      return 0;
31  }
32  // 编译通过，但是运行结果错误
33  // 预期结果 2.3
34  // 实际结果 -546132160.000000
```

C++

C++ 是一种通用的编程语言，支持面向对象、过程化和泛型编程。C++ 是 C 语言的扩展，增加了面向对象的特性，如类和对象。C++ 由 Bjarne Stroustrup 于 1979 年在贝尔实验室设计开发，最初称为“C with Classes”，后来在 1983 年更名为 C++。现代 C++ 指的是 C++11 及其之后的版本。

推荐阅读：《C++ Primer Plus》

https://en.cppreference.com/w/

C++ 基础

Hello, World!

代码块

```cpp
#include <iostream> // 引入流输入输出库

using namespace std;//命名空间

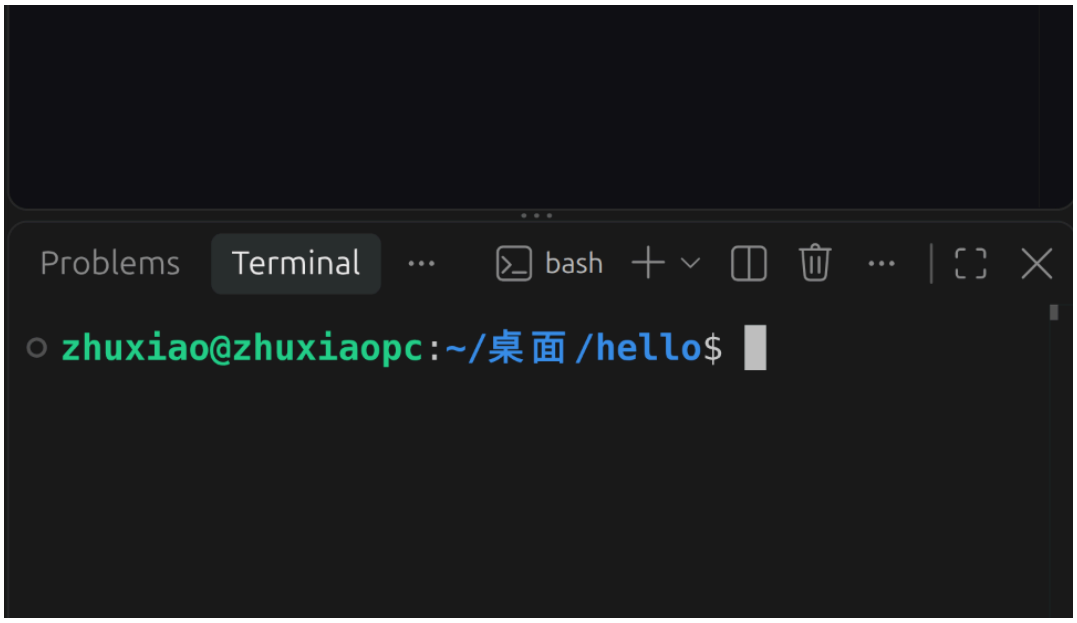
int main() {

// 使用标准命名空间
// 主函数
cout << "Hello, World!" << endl

// 输出 Hello, World!, endl 为换行符return 0;
// 返回 0，表示程序正常结束
}
```

编译：

要运行这段代码，我们必须要先编译，也就是我们前半节课讲的，写完代码后打开Terminal(没看到可以ctrl+`呼出)



输入:

代码块

```
mkdir build  //新建build文件夹
g++ 文件名.cpp -o /build/输出文件名  //编译
./输出文件名 //运行程序
```

为什么要新建build文件夹?

生成的 .o、可执行文件不会弄脏源码目录

项目变大、用 CMake 时，构建产物统一放 build/，方便一键删除重来

命名空间

为什么要有命名空间?

命名空间 = 给名字加“姓氏”，用来防止重名冲突

例如:

学校里有两个叫“张伟”的同学，你喊“张伟”没人知道叫谁。但你说“一班的张伟”“二班的张伟”就清楚了。

- 班级 = 命名空间
- 张伟 = 里面的变量/函数名
- 一班::张伟 = 用 ::（作用域运算符）指明“是哪家的”

注意:

using namespace std; 在练习代码里很常见，但在头文件或大项目里**不要写**，会把很多常见名字（count、max、swap…）放进全局，反而制造冲突。

流

这个是在c++里新引入的概念

流（stream）是程序和数据源/目标之间的一条“数据管道”，数据像水一样按顺序流过。你不用关心数据具体从键盘、文件还是网络来，只管从“流”里读或往里写。

这些是C++里四个标准流，其中最常用的是cin和cout

C++ 里四个标准流

流	含义	方向
cin	标准输入（键盘）	进程序
cout	标准输出（屏幕）	出程序
cerr	标准错误（不缓冲，立即输出）	出程序
clog	标准错误（带缓冲）	出程序

在程序开头的#include<iostream>就是引入流

其中包含两个运算符——<< 和>>

<< 插入运算符：把数据“倒进”输出流 → cout << "hi" << endl;

>> 提取运算符：从输入流“舀出”数据 → cin >> x;

都返回流本身，所以能链式写：cout << a << b << c;

数据类型、运算符、流程控制、注释

这些内容与 C 语言类似，不再赘述。C++ 新增了一些数据类型，常用的有：

bool：布尔类型，取值为 true 或 false。作为逻辑运算返回值类型。

string：字符串类型，用于存储文本。需要引入头文件 #include <string>。

nullptr：空指针类型，用于表示空指针。

auto：自动类型推导，用于自动推导变量的类型。

decltype：类型推导，用于获取表达式的类型。

C++ 新增了一些运算符，常用的有：

::：作用域运算符，用于访问命名空间、类、对象的成员。

new、delete：用于动态内存分配和释放。

typeid：用于获取表达式的类型信息。

const_cast、static_cast、dynamic_cast、reinterpret_cast：用于类型转换。

C++ 对 for 循环进行了增强，支持使用迭代的方式遍历容器：

代码块

```
#include <vector>

using namespace std;

int main() {

    vector<int> vec = {1, 2, 3, 4, 5};

    for (auto &it : vec) {

        cout << it << " ";

    }
//1 2 3 4 5
    return 0;
// 使用迭代的方式遍历容器}
```

保留字

相较于 C, C++ 新增了非常多的保留字，比较常见的：

new、delete：用于动态内存分配和释放。

class、public、private、protected：用于定义类和访问权限。

virtual、override、final：用于虚函数和多态。

namespace、using：用于命名空间。

template、typename：用于模板和泛型编程。

try、catch、throw：用于异常处理。

constexpr、static_assert：用于编译时常量和断言。

operator、friend：用于运算符重载和友元函数。

函数重载

与 C 语言不同，C++ 支持函数重载，即可以定义多个同名函数，只要它们的**参数列表不同**即可。

函数重载可以根据参数的个数、类型、顺序进行区分。

代码块

```
int add(int a, int b) {

    return a + b;

}
double add(double a, double b) {

    return a + b;

}
int add(int a, int b, int c) {

    return a + b + c;

}

```

运算符重载

C++ 支持运算符重载，即可以对运算符进行重定义，使其适用于自定义类型。运算符重载是通过函数重载实现的，函数名为 operator 后接运算符。

代码块

```
class Complex {
```

```
2
3   public:
4
5       Complex(double r = 0.0, double i = 0.0) : real(r), imag(i) {}
6       //定义了一个包含虚部和实部的数据类型
7
8       Complex operator + (const Complex &c) {
9       //数据类型 operator 重载的运算符（这里是+，c++中大部分运算符都可以进行重载）
10      //括号里的内容：
11      //const:承诺函数内部不修改 c。既保护实参，又让函数能接收 const 对象和临时对象；
12      //不加的话，const Complex x; x + y 会编译失败
13      //&是用引用的方式传参数，不然会复制整一个参数，效率太低且可以直接操作原对象
14      return Complex(real + c.real, imag + c.imag);
15      //返回值就是虚部相加和实部相加
16  }
17
18  private:
19
20      double real, imag;
21
22  };
```

引用

C++ 支持引用，引用是一个变量的别名，通过引用可以直接访问变量的值。引用在定义时需要初始化，初始化后不能再引用其他变量。

代码块

```
1   int a = 10;
2
3   int &b = a; // b 是 a 的引用
```

此时，b 是 a 的引用，对 b 的操作等同于对 a 的操作。对引用的操作会直接影响到原变量。

注意和指针的区别：引用是变量的别名，指针是变量的地址。

刚接触可能会比较难理解，这里举一些通俗的例子：

引用 = 外号/别名

指针 = 写着地址的纸条

&的意思是取地址，这里就是a和b的地址是同一个，相当于就是两个名字指向同一个人

- 引用是“给变量起别名”，本质还是那个变量本身。
- 指针是“保存变量地址的另一块内存”，通过地址间接访问变量。

Lambda

Lambda 就是“随手写的一个小函数”，不用起名字，用完就扔。

比如要一个“加 1”的函数：

普通写法：

```
代码块
1   int add1(int x) { return x + 1; }
```

Lambda 写法：

```
代码块
1   auto add1 = [](int x) { return x + 1; };
2
```

用起来一样：add1(5) → 6。

语法就三块

[](int x) { return x + 1; }

↑ ↑ ↑

[] 参数 函数体

[]是 lambda 的标志，看到 [] 就知道是 lambda。

为什么需要它?

有些函数只用一次，专门起名字很麻烦。比如排序：

代码块

```
std::sort(v.begin(), v.end(), [](int a, int b) {
    return a > b;          // 降序
});


```

原本sort为升序排列，现在后面加了一个比较器，就可以省去写另外一个函数的操作。

Lambda = [] + 参数 + 函数体，本质是个“就地写的匿名函数”，常用于回调、排序、遍历这种只用一次的地方。

异常处理

C++ 支持异常处理，通过 try、catch、throw 关键字实现。try 用于尝试执行可能发生异常的代码，catch 用于捕获异常并处理，throw 用于抛出异常。

代码块

```
try {
// 尝试执行可能发生异常的代码
    if (error) {

    throw "Error"; // 抛出异常

    }
}
catch (const char *msg) {

    // 捕获异常并处理cout << msg << endl;

```

```
}
```

多线程

对比进程，线程的优势：

可以利用多核并行处理，大大提升效率

如何创建线程：

代码块

```cpp
#include <iostream>
#include <thread>

void hello() {
    std::cout << "Hello from thread\n";
}

int main() {
    std::thread t(hello);    // 启动线程执行 hello
    t.join();                // 等线程结束
}

```

以上代码就创建了一个线程，实际代码中我们常常用多个线程并行处理来提高效率

锁：

多线程共享数据时，保证同一时刻只有一个线程进入，否则会造成数据竞争，例如：

代码块

```cpp
int counter = 0;
// 两个线程各 ++ 100000 次
counter++;   // 内部是"读→加→写"三步，可能被打断

```

基本用法：

代码块

```cpp
#include <mutex>
std::mutex mtx;

void add() {
    mtx.lock();       // 上锁
    ++counter;        // 临界区：同一时刻只有一个线程能进
    mtx.unlock();     // 解锁
}


```

课后实践：

写一个c++程序实现虚数减法（用重载运算符），编译并运行成功。